A fast learning algorithm for deep belief nets
&
Deep learning

BCSC 스터디 5주차

Part 1. Hinton et al. (2006)
Part 2. LeCun et al. (2015)

Deep Network 학습의 두 관점

Part 1 · Generative Model	Part 2 · Discriminative / Representation Learning
관측 v에서 숨은 원인 h를 어떻게 추론하는가?	예측의 오차가 각 층의 가중치와 어떻게 연결되는가?
Posterior와 Likelihood	Loss와 Gradient
RBM · CD · Layer-wise Training	Forward Pass · Chain Rule · Backpropagation

> 한쪽은 **확률분포 학습**, 다른 한쪽은 **손실의 Gradient 계산 중심**.

PART 1 — A Fast Learning Algorithm for Deep Belief Nets

Difficulty in Deep Neural Network

- 가중치의 수와 은닉층의 증가
- 생성모형 ← Hidden State의 Posterior(사후확률) 추론이 복잡함. (Explaining Away)

Complementary Prior, Inference

Explaining Away → Posterior 복잡해짐.

https://widgetstation.com/embed?t=0d3d8a3fa3e094235af552f21fb038558ba669ecc0eadc0d2dc5b71ff1cc74a6&lang=ko

---

Complementary Prior, Inference — 2

$$p(s_i = 1) = \frac{1}{1+\exp(-z_i)}$$

$$z_i = b_i + \sum_j s_j w_{ij}$$

해결: Complementary Prior를 통해

$$p(h \mid v) = \prod_j p(h_j \mid v)$$

처럼 Factorial Posterior를 구성.

RBM Learning & Contrastive Divergence

RBM의 Maximum Likelihood Gradient

$$\frac{\partial \log p(v^0)}{\partial w_{ij}} = \langle v_i^0 h_j^0 \rangle - \langle v_i^\infty h_j^\infty \rangle$$

실제 데이터 공동 활성도 – 모델 평형분포 공동 활성도

Contrastive Divergence의 목적 함수

$$CD_n = KL(P^0 \| P_\theta^\infty) - KL(P_\theta^n \| P_\theta^\infty)$$

P^0: 데이터분포 / P_θ^n: n회 Gibbs Sampling / P_θ^∞: 평형분포

RBM Learning & Contrastive Divergence — 2

- RBM은 같은 층 내 연결 X → 조건부 추론이 쉬운 구조.
- 학습을 통해 실제 데이터의 공동 활성과 모델에서 생성되는 공동 활성의 차이를 감소시키지만 평형분포까지 Gibbs Sampling(Marcov chain Monte-Carlo algorithm의 일종)은 시간이 오래 걸림.
- 실제 데이터에서 출발해 짧게 샘플링한 분포를 대신 사용하는 Contrastive Divergence(대조 발산)가 필요.

Greedy Layer- wise Learning

Learning process

1. 첫 RBM을 원본 데이터로 학습시킴.
2. 아래층의 가중치를 고정.
3. Hidden Representation을 다음 RBM의 데이터로 사용.
4. 상위 RBM을 차례로 추가.

단순한 경험적 요령이 아니라, 다음 슬라이드의 Variational Bound를 통해 이 절차를 정당화하려 함.

Variational Bound

Greedy Layer-wise Learning이 전체 Generative Model을 개선할 수 있는 근거

Energy 정의

$$E(v^0, h^0) = -\left[\log p(h^0) + \log p(v^0 \mid h^0)\right]$$

Variational Bound - 2

$$\log p(v^0) \geq \sum_{\text{all } h^0} Q(h^0 \mid v^0)\left[\log p(h^0) + \log p(v^0 \mid h^0)\right] - \sum_{\text{all } h^0} Q(h^0 \mid v^0) \log Q(h^0 \mid v^0)$$

세 항의 의미

- $\log p(h^0)$: 상위 가중치가 정의하는 첫 Hidden Layer의 **Prior**
- $\log p(v^0 \mid h^0)$: Hidden State에서 관측 데이터를 생성하는 **Likelihood**
- $-\sum Q \log Q$: 추론분포 $Q(h^0 \mid v^0)$의 **Entropy**
- $Q(h^0 \mid v^0)$: 첫 Hidden Layer의 상태에 대한 추론분포

핵심 조건: $Q(h^0 \mid v^0) = p(h^0 \mid v^0)$일 때, 그리고 그때에만 하한이 실제 $\log p(v^0)$와 같아짐.

Variational Bound — 3

아래층과 추론분포 Q를 고정하면 남는 학습 항(최적화해야 할 부분)

$$\sum_{\text{all } h^0} Q(h^0 \mid v^0) \log p(h^0)$$

> 첫 Hidden Layer의 상태 h^0가 $Q(h^0 \mid v^0)$의 확률로 나타나는(true posterior와 같음) 데이터셋에 대해 **상위 RBM의 Log Likelihood를 최대화**하는 것과 같음. RBM을 한 층씩 학습할 때, 수학적으로는 전체 모델의 성능이 나빠지지 않도록 할 수 있지만, 실제 사용하는 근사 알고리즘에서는 그 결과를 보장할 수 없음.

Fine-tuning, Softmax & MNIST

Greedy Pre-training → Contrastive Wake-Sleep Fine-tuning

- **Up-pass:** 실제 데이터에서 Hidden State를 추론하고 Generative Weight를 조정.
- **Down-pass:** 생성된 상태를 통해 Recognition Weight를 조정.

Label Unit의 Softmax(활성화 함수 중 하나)

$$p_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Fine-tuning, Softmax & MNIST — 2

단계	MNIST Test Error
Greedy Pre-training	2.49%
Up-down Fine-tuning	1.39%
최종 전체 데이터 학습	**1.25%**

층별 학습으로 초기값을 얻은 후 전체 모델을 Contrastive Wake-Sleep으로 조정.

입력 이미지와 레이블의 결합분포를 학습하기 때문에 레이블 유닛에는 Softmax가 사용됨.

Summary

- 뉴런의 상태를 0 or 1로 두고 활성 확률을 함수로 정의.
- Complementary prior로 은닉변수들이 관측 데이터에 대해 독립이 되도록 함.
- 초기 모델에서 여러 층 가중치가 동일하다고 가정, RBM과 infinite directed model의 동등성 이용.
- 상위의 RBM을 정확히 학습해야 모델 개선이 보장됨. (수학적 엄밀함 부족)
- Gibbs sampling 사용으로 오래걸리고 복잡해짐.
- 분류가 어려울 때 데이터를 순차적으로 선택하여 관찰.

Transition — Two Learning Strategies

생성모형의 학습에서 Gradient 기반 신경망 학습으로

Hinton et al. (2006)	LeCun et al. (2015)
Posterior 추론 및 생성확률	Representation과 손실
RBM, CD, Variational Bound	Chain Rule, Backpropagation
층별 초기학습과 Up-down	다층망의 Gradient 전파

> **다음 질문:** 분류 손실이 정해졌을 때, 그 손실을 줄이기 위한 모든 가중치의 Gradient는 어떻게 계산하는가?

PART 2 — Deep Learning

LeCun, Bengio & Hinton, 2015 · Representation Learning과 Backpropagation

Representation Learning

분류의 핵심은 원본 입력을 구분하기 쉬운 Representation으로 바꾸는 것.

$$h = f(Wx + b)$$

- 선형 조합 $Wx + b$만 반복하면 전체도 Affine Mapping이 됨.
- Nonlinear Activation이 새로운 특징과 비선형 결정 경계를 가능하게 함.
- 높은 층은 입력의 Task-relevant Structure를 표현함.

원본 공간에서 뒤엉킨 두 클래스 → 은닉층을 거치며 분리.

Forward Propagation

각 층의 계산

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

z: 활성화 함수 이전 값 / a: 활성화 함수 이후 값

> Input → Weighted Sum → Nonlinearity → Hidden Representation → Output

각 층에서 이전 층의 출력을 가중합하고 활성화 함수를 적용하는 과정을 반복, 새로운 representation과 최종 prediction을 얻음.

f가 미분 가능하거나 적절한 Subgradient를 사용할 수 있어야 이후 Backpropagation을 적용 가능.

Loss & Gradient Descent

예측이 틀린 정도를 Loss L로 정의한다.

$$L = \frac{1}{2}(\hat{y} - y)^2$$

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

- **Gradient Descent:** 기울기를 이용해 가중치 수정.
- **Backpropagation:** 그 기울기를 효율적으로 계산.

Deep network의 모든 가중치에 대한 미분은 어떻게 계산하는가가 핵심.

Chain Rule

함수의 합성을 미분하는 기본 원리

$$x \xrightarrow{f} h \xrightarrow{g} L$$

$$\frac{dL}{dx} = \frac{dL}{dh}\frac{dh}{dx}$$

Backpropagation — Output Layer

두 층짜리 예시

$$z_1 = w_1 x + b_1, \qquad h = \sigma(z_1)$$

$$z_2 = w_2 h + b_2, \qquad \hat{y} = \sigma(z_2)$$

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Backpropagation — Output Layer

출력층의 Loss Gradient

$$\delta_2 = \frac{\partial L}{\partial z_2}$$

$$\delta_2 = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

Backpropagation — Hidden Layer

은닉층까지 Chain Rule 적용

$$\delta_1 = \frac{\partial L}{\partial z_1}$$

$$\delta_1 = \delta_2 w_2 h(1 - h)$$

왜 $h(1-h)$인가?

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

$$\sigma'(z_1) = h(1 - h)$$

다음 층의 Gradient × 연결 가중치 × 현재 층의 국소 미분.

h가 0이나 1에 매우 가까우면 이 도함수가 작아져 Gradient Vanishing 일어날 수 있음.

General Backpropagation in Matrix Form

모든 뉴런에 대해 역전파

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot f'(z^{(l)})$$

가중치 행렬의 Gradient: Outer Product(바깥곱)

$$\nabla_{W^{(l)}} L = \delta^{(l)} (a^{(l-1)})^T$$

- $W^T\delta$: 다음 층에서 오는 모든 Gradient 합산.
- $\odot$: 활성화 함수 도함수의 원소별 곱.
- Outer Product: 현재 층 모든 연결 가중치의 Gradient 행렬.

General Backpropagation in Matrix Form — 2

하나의 뉴런이 여러 다음 층 뉴런에 연결되면 손실의 영향이 여러 경로를 통해 입력됨.

전치행렬 W^T를 곱하는 과정이 이 영향들을 더함. (원래 방향과 반대방향으로 계산한다는 의미)

활성화 함수의 국소 미분은 원소별로 적용. 그 결과 δ를 얻으면 이전 층 출력벡터와의 Outer Product로 모든 연결 가중치의 Gradient를 동시에 구할 수 있음. 여기에서 사용되는 것은 3차원 Cross Product가 아니라 행렬 형태의 Outer Product.

CNN

ConvNet의 핵심 구조: 4가지 원리

- **Local Connections:** 이전 층의 작은 영역(Local Patch)을 관찰.
- **Shared Weights:** 같은 Filter로 여러 위치의 패턴을 검출.
- **Pooling:** 작은 위치 이동이나 변형에 대한 민감도를 낮추고 표현 크기를 줄임.
- **Many Layers:** 단순 특징을 결합해 복잡한 특징을 학습.

Convolution → ReLU → Pooling → Higher-level Feature Map

> 이미지의 국소 상관관계와 패턴의 위치 독립성을 네트워크 구조에 반영.

학습된 필터의 가중치는 Backpropagation으로 갱신.

CNN — Hierarchical Representation & Results

학습되는 Representation의 계층

> Edge → Motif → Object Part → Object

성과

- 손글씨 및 문서 인식, 얼굴·보행자 탐지, 생물학 영상 분할.
- **2012 ImageNet:** 기존 방법의 오류율을 거의 절반으로 감소시킨 Deep ConvNet의 성과.
- 2015년 기준 컴퓨터 비전에서 CNN 활용이 빠르게 확산.

낮은 층의 Feature Map은 Edge 같은 단순한 패턴을 감지 → 위쪽 층에서는 패턴들의 조합이 물체의 부분과 전체를 표현 →

사람이 Edge Detector와 Object Feature를 일일이 설계할 필요를 줄임.

Deep Learning — Advantages

장점	의미
Automatic Feature Learning	사람이 특징을 직접 설계하는 부담을 줄임
Hierarchical Representation	여러 층이 점차 추상적인 특징을 학습
Selectivity & Invariance	분류에 필요한 차이는 감지하고 무관한 변동에는 상대적으로 둔감
End-to-end Learning	여러 층의 가중치를 Backpropagation으로 함께 학습 가능

> **데이터에 적합한 Representation을 학습.**

Deep Learning — Limitations & Future Directions

이 논문상의 한계

문제 또는 미해결 과제	논문에서 제시한 방향
Labelled Data가 적은 학습 상황	Unsupervised Pre-training 및 Representation Learning 활용
RNN의 Long-term Dependency 학습 어려움	LSTM과 Memory Module 도입
수동적인 이미지 인식의 한계	Attention, RNN, Reinforcement Learning을 결합한 Active Vision
복잡한 Reasoning의 어려움	Representation Learning과 Reasoning의 결합

> **주의:** 이는 논문이 설명한 당시의 문제와 연구 방향으로, CNN이나 모든 딥러닝 모델에 공통으로 적용되는 불가능성 정리가 아님.

CONCLUSION

Deep Networks를 학습하는 두 관점

Part 1 · Hinton (2006)

Complementary Prior
↓
RBM & Contrastive Divergence
↓
Greedy Layer-wise Learning
↓
Up-down Fine-tuning

Part 2 · LeCun et al. (2015)

Representation Learning
↓
Forward Pass & Loss
↓
Chain Rule & Backpropagation
↓
Weight Update

